Chapter 12 Merge sort

Divide-and-Conquer

Prompt 1: What are the three steps of the divide-and-conquer algorithmic paradigm?

Answer:

Divide-and-conquer is a recursive design pattern consisting of three main steps:

Divide:

- If the input size is small enough (base case), solve it directly.
- Otherwise, split the input into two or more smaller, disjoint subsets.

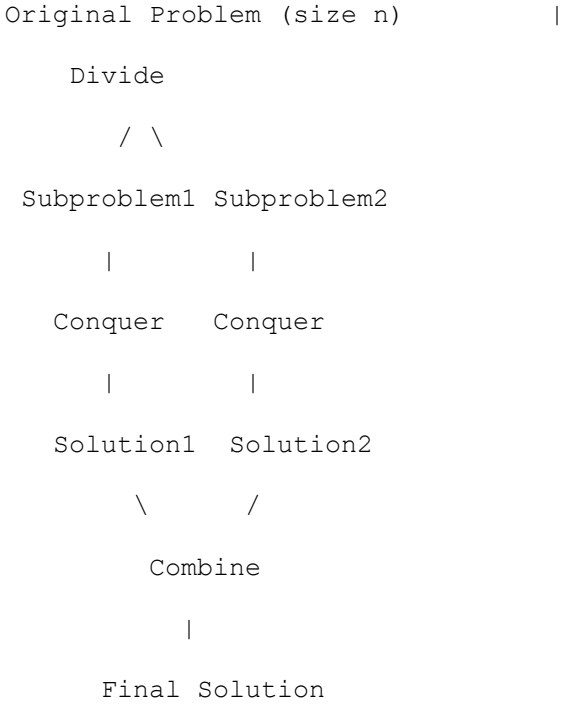
Conquer:

- Solve each of these subsets recursively using the same algorithm.

Combine:

- Merge the solutions of the subproblems to form a solution for the original problem.

Visual Example:



Step 2: Applying Divide-and-Conquer to Sorting (Merge-Sort)

Prompt 2: How does merge-sort implement the divide, conquer, and combine steps?

Answer:

Divide:

- If the sequence has 0 or 1 elements → already sorted, return it.
- If the sequence has ≥2 elements → split into two halves:
  - S_1 = first $n/2$ elements
  - S_2 = remaining elements

Conquer:

- Recursively sort S_1 and S_2.

Combine:

- Merge the sorted sequences S_1 and S_2 to produce a single sorted sequence.

Key Idea: The most "complex" step is **combining** because we have to merge two sorted sequences efficiently.

Step 3: Understanding the Merge Process

Prompt 3: How do we merge two sorted sequences?

Answer:

To merge sequences S_1 and S_2 (both already sorted):

1. Initialize two pointers: `i = 0` for S_1, `j = 0` for S_2.
2. Compare elements $S_1[i]$ and $S_2[j]$:
   - If $S_1[i] \leq S_2[j]$ → add $S_1[i]$ to the merged array, increment `i`.
   - Else → add $S_2[j]$ to the merged array, increment `j`.
1. Repeat until one sequence is exhausted.
2. Copy the remaining elements of the non-empty sequence to the merged array.

Time Complexity of Merge Step: $O(n)$, because we look at each element exactly once.

Step 4: Step-by-Step Visualization

Let's visualize merge-sort for the sequence:

```
S = [85, 24, 63, 45, 17, 31, 96, 50]
```

1. Divide Step

```
S1 = [85, 24, 63, 45]   S2 = [17, 31, 96, 50]
```

2. Conquer Step

```
S1 → [85, 24] and [63, 45]

S2 → [17, 31] and [96, 50]
```

3. Recursive Conquer

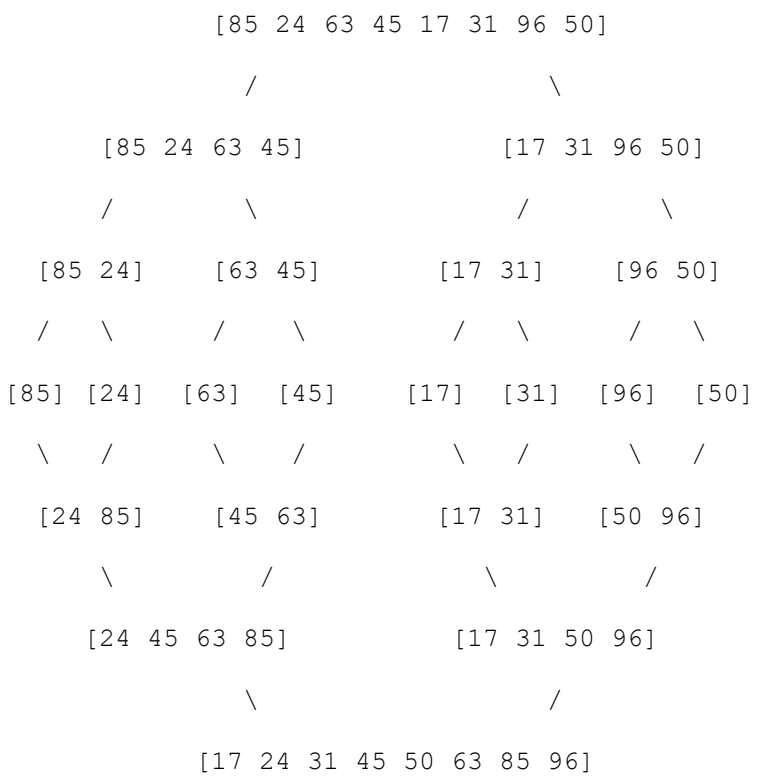
```
S1 left → [85] and [24] → merge → [24, 85]

S1 right → [63] and [45] → merge → [45, 63]

S1 merge → [24, 45, 63, 85]

S2 left → [17] and [31] → merge → [17, 31]

S2 right → [96] and [50] → merge → [50, 96]

S2 merge → [17, 31, 50, 96]
```

4. Combine

```
Final merge: [24, 45, 63, 85] + [17, 31, 50, 96] → [17, 24, 31, 45, 50, 63,
85, 96]
```

Step 6: Merge-Sort Tree Diagram

Here's a simplified merge-sort tree for the above sequence:



- Leaves are base cases (single elements).
- Internal nodes represent merging sorted subarrays.

Array-Based Implementation of Merge-Sort

Step 1: Understanding the Merge Function

Prompt 1: What does the `merge` function do in merge-sort, and how does it work?

Answer:

The `merge` function combines **two already sorted sequences** `S1` and `S2` into a single sorted sequence `S`.

How it works (line by line):

```
def merge(S1, S2, S):

    i = j = 0                # i: index in S1, j: index in S2

    while i + j < len(S):    # Continue until all elements are copied

        if j == len(S2) or (i < len(S1) and S1[i] < S2[j]):

            S[i + j] = S1[i]  # Copy element from S1

            i += 1

        else:

            S[i + j] = S2[j]  # Copy element from S2

            j += 1
```

- **i + j** keeps track of the position in the final array `S`.
- Compare **current elements** of `S1[i]` and `S2[j]`.
    - Copy the smaller one into `S[i + j]`.
- If one array is exhausted, copy all remaining elements from the other array.

Time Complexity:

- Each element is copied exactly once → `O(n)` where `n = len(S1) + len(S2)`.

Prompt 2: Can we visualize the merging step?

Answer:

Yes! Suppose:

```
S1 = [9, 19, 25]

S2 = [2, 10, 18]

S = [_, _, _, _, _, _]  # empty output array
```

Step-by-step merge:

i	j	i+j	S[i+j]	Action	S after step
0	0	0	2	S2[0] < S1[0]	[2, _, _, _, _, _]
0	1	1	9	S1[0] < S2[1]	[2, 9, _, _, _, _]
1	1	2	10	S2[1] < S1[1]	[2, 9, 10, _, _, _]
1	2	3	19	S1[1] < S2[2]	[2, 9, 10, 19, _, _]
2	2	4	18	S2[2] < S1[2]	[2, 9, 10, 19, 18, _]
2	3	5	25	S1[2] left	[2, 9, 10, 19, 18, 25]

✅ Result after merge: `[2, 9, 10, 18, 19, 25]`

Step 2: Understanding Array-Based Merge-Sort

Prompt 3: How does the array-based `merge_sort` function work?

Answer:

```
def merge_sort(S):
    n = len(S)
    if n < 2:
        return              # Base case: list of 0 or 1 elements is sorted
    mid = n // 2
    S1 = S[0:mid]           # Copy first half
    S2 = S[mid:n]           # Copy second half
    merge_sort(S1)          # Recursively sort first half
    merge_sort(S2)          # Recursively sort second half
    merge(S1, S2, S)        # Merge sorted halves back into S
```

Step-by-step:

Base case:

- If the list has 0 or 1 elements → already sorted.

Divide:

- Split `S` into two halves (`S1` and `S2`).

Conquer:

- Recursively call `merge_sort` on both halves.

Combine:

- Merge the sorted halves back into the original list `S` using the `merge` function.

Key Points:

- Uses **extra space** for `S1` and `S2` → not in-place.
- Recursion depth = `log n`.
- Merge cost at each level = `O(n)` → overall complexity = `O(n log n)`.

Prompt 4: Can we visualize the entire array-based merge-sort on an example list?

Answer:

Example list:

```
S = [25, 18, 19, 22, 9, 10]
```

Step 1: Divide

```
S1 = [25, 18, 19]
```

```
S2 = [22, 9, 10]
```

Step 2: Recursive Conquer

```
S1 → [25, 18] and [19]
```

```
S2 → [22, 9] and [10]
```

Step 3: Further Conquer

```
S1: [25, 18] → [25], [18] → merge → [18, 25]
```

```
S1 merge with [19] → [18, 19, 25]
```

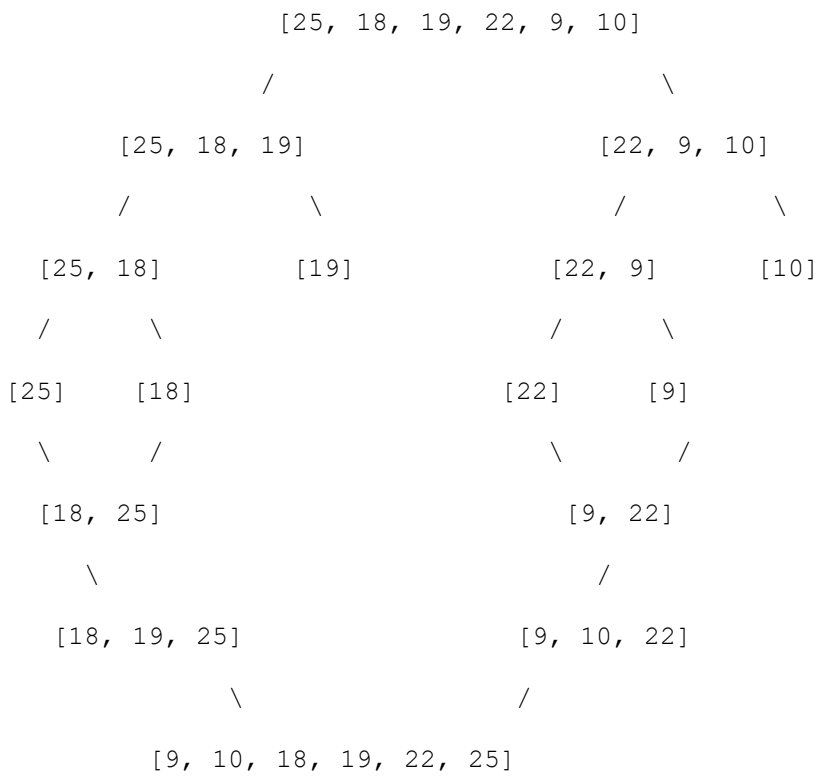
```
S2: [22, 9] → [22], [9] → merge → [9, 22]
```

```
S2 merge with [10] → [9, 10, 22]
```

Step 4: Final Merge

```
Merge S1 = [18, 19, 25] with S2 = [9, 10, 22] → [9, 10, 18, 19, 22, 25]
```

Diagram of Merge-Sort Tree for S = [25, 18, 19, 22, 9, 10]:



The Running Time of Merge-Sort

Step 4: Total Running Time

Prompt 4: How do we compute the total time for merge-sort?

Answer:

- Height of merge-sort tree = $log_2 n$ (because the sequence is split in half each time).
- Levels = 0,1,2,..., $log\, n \rightarrow$ total of $log\, n + 1$levels.
- Time per level = O(n)

Total time:

$O(n) \cdot (\log n+1)=O(n\log n)$